Contributing to Open Factory Studio

Thanks for helping out. This page explains how to get a build going and the few rules every change has to follow.

Getting started

1. Build the project as explained in the Building document.
2. Run the tests with ctest --preset <preset>. They all have to pass before you open a pull request.
3. Have a look at the "How the code is organized" document to find your way around.

Rules

Behavior

Open Factory Studio should behave like Fighter Factory Studio 3.7: same menus, shortcuts, panels, options, file output and way of working. If you change something people can see, check it against Fighter Factory. The settings files (config.xml, shortcuts.xml, history.xml) keep the same XML layout, so old settings can still be imported.

New engines (IKEMEN GO for example) go in their own plugin in src/engines/<name>, with their data in data/engines/<name>. See the Engine plugins document.

Files from other programs (important)

- **Never commit files from Fighter Factory, M.U.G.E.N or any other program.** That means no icons, help pages, syntax files, presets, style sheets, sprites, sounds, characters (Kung Fu Man too), fonts or stages. Everything in data was made for this project, or comes from an openly licensed source listed in the third party notices.
- **Never copy code from closed source tools**, and don't decompile them.
- Tests can't rely on game files in the repository. Make the test files in code (see tests/fixtures). Tests that need real files read them from the folders in OFS_CORPUS_KFM and OFS_CORPUS_MUGEN, and are skipped when those aren't set.
- Code from open source projects is fine if its license works with the GPL-3.0. Add the library to the third party notices with its license text.

Code style

- The code is C++20 with Qt 6. Format it with clang-format 18 (the settings are in .clang-format), for example by running git clang-format before you commit. The CI build checks it.
- Namespaces: OFS for the core, OFS::Formats for file formats and OFS::Mugen for the M.U.G.E.N plugin.
- QT_NO_KEYWORDS is on, so use Q_SIGNALS, Q_SLOTS and Q_EMIT.
- Includes start from src, like #include "core/project/Project.h".
- src/formats must not use Qt, so it can be fuzzed and used by the command line tools.

- New features need tests: Catch2 for code without widgets, QtTest for widgets.

Translations

The interface is written in American English and translated with Qt Linguist. Each language has a folder data/languages/<code> (for example pt-br) with an info.xml that names the language and the translator, and an ofs.ts file with the translations:

4. Make the folder and its info.xml (copy data/languages/en-us/info.xml and change it).
5. Make the translation file from the code: lupdate -locations relative src -ts data/languages/<code>/ofs.ts.
6. Translate it in Qt Linguist. The build turns it into ofs.qm in the data folder. After code changes, the update_translations target updates every .ts file.

The language is picked in Options > General. Qt's own texts (standard buttons and dialogs) come from Qt's translations for the same language.

Commits and pull requests

- Keep each commit to one thing, with a short subject line like "Add LZ5 decoder".
- Pull requests need a green CI build and a few words on what changed and how you tested it.
- Add changes people will notice to the changelog.

By contributing you agree that your work is licensed under the GPL-3.0.